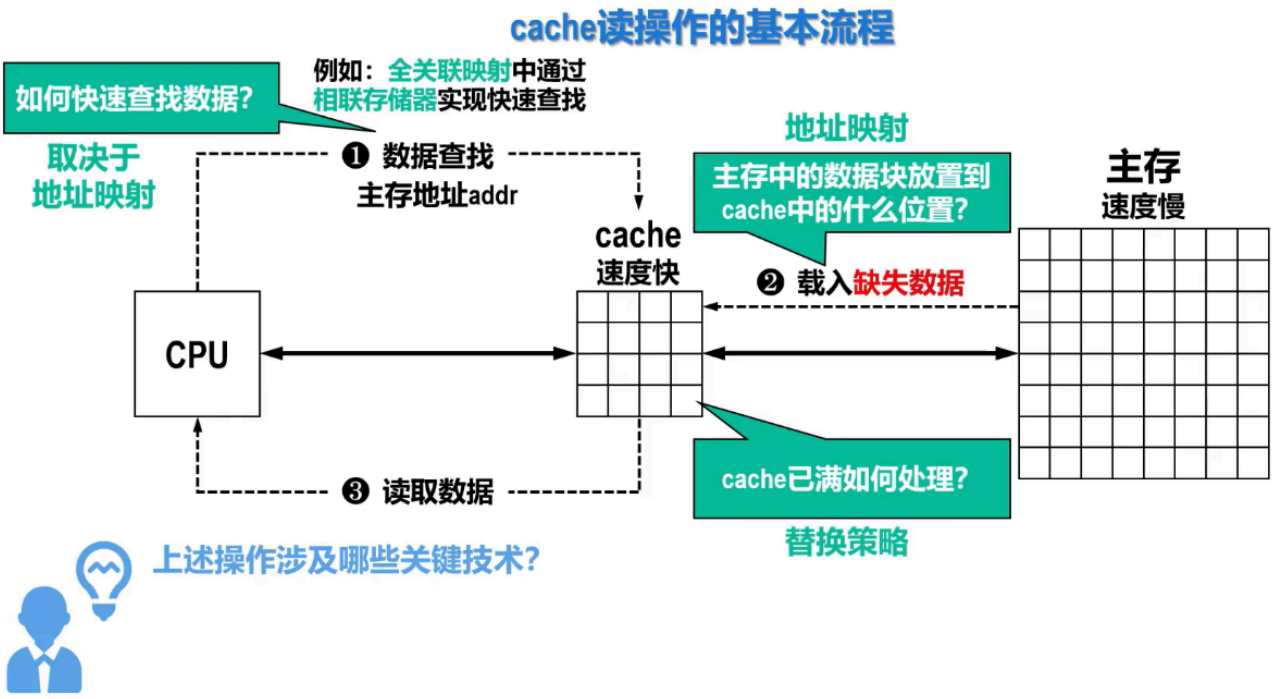
4.22 Cache——读写流程

考纲定位：计算机组成原理 · 第 4 章 存储器（4.22） | 重要度 ★★★★★ | 常见考法：选择题/大题（读流程、写回/写穿、写分配/非写分配、脏位）

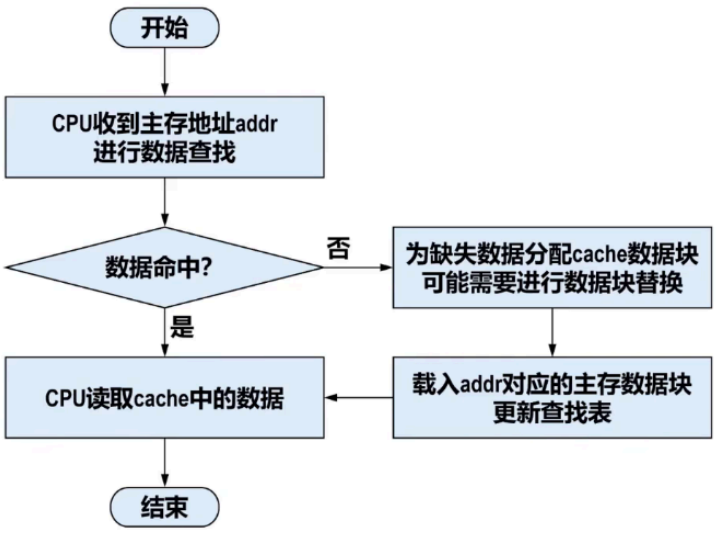
一句话核心：读操作先查 Cache，命中直接取，缺失调块；写操作分“写命中”和“写缺失”，分别选择写回/写穿、写分配/非写分配

一、读流程

读操作涉及的问题



读操作流程图



- 读命中访问时间最短。
- 构建cache系统时，应尽可能提高命中率，以提升读操作性能。
  - 通过较好的替换策略，将经常访问的热数据保留在cache中，将不经常访问的冷数据淘汰，来充分利用时间局部性，以提高命中率。
  - 数据缺失时，会将缺失数据所在数据块中的其他数据一起载入，这种预读策略可以充分利用空间局部性来提高顺序访问的命中率。

步骤

1. CPU 给出主存地址
2. 用地址中的索引找到 Cache 行
3. 比较 Tag 与有效位
4. 命中：直接读出数据
5. 缺失：从主存调入整个块，替换某行，再读出数据

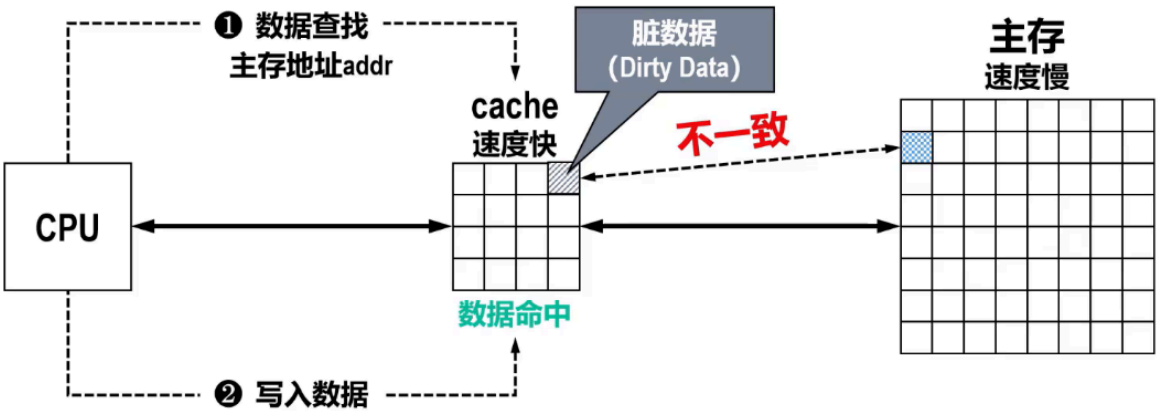
易错

- 缺失时调入的是“块”，不是单个字
- 有效位为 0 时，即使 Tag 相同也不能算命中
- 读缺失可能触发替换算法；直接映射只能替换固定行

二、写流程

2.1 命中缓存时

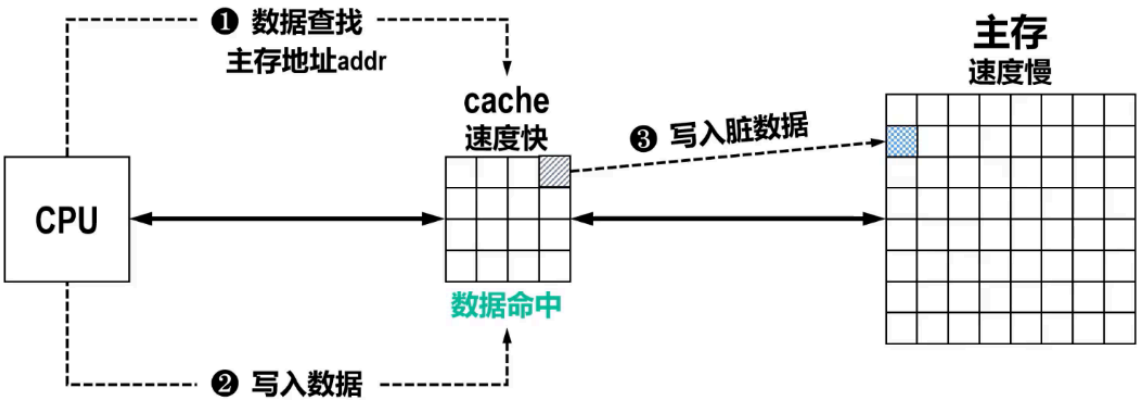
写回策略（Write-Back）



写回(Write-back)策略 **此时写入操作结束，这种方式响应速度最快，但会产生不一致性。**

- 写命中时只修改 Cache，不立即写主存
- 该行设置脏位（dirty bit），表示与主存不一致
- 该行被替换时，若脏位为 1，先写回主存
- 优点：减少主存写次数；缺点：控制复杂，需要脏位

写穿策略（Write-Through）



写穿(Write-through)策略 **此时还需要将脏数据写入慢速的主存中才能结束，这种方式响应速度较慢，但没有脏数据产生。**

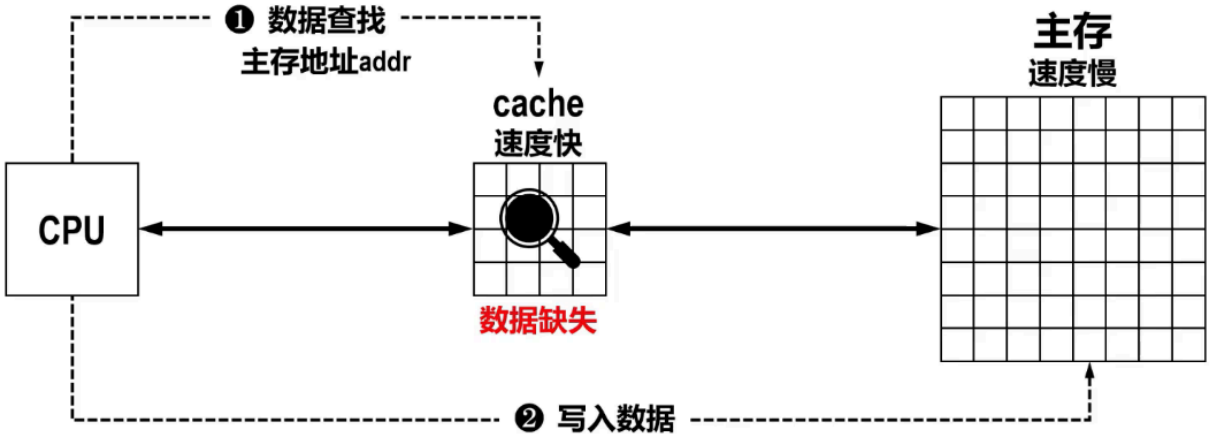
- 写命中时同时写 Cache 和主存
- 不需要脏位，控制简单
- 缺点：每次写都访问主存，写带宽压力大
- 常配合写缓冲（write buffer）减少等待

易错

- 写回不是“不写主存”，而是“延迟到替换时写”
- 写穿不是“只写主存”，Cache 也要更新
- 脏位只在写回策略中使用

2.2 未命中缓存时

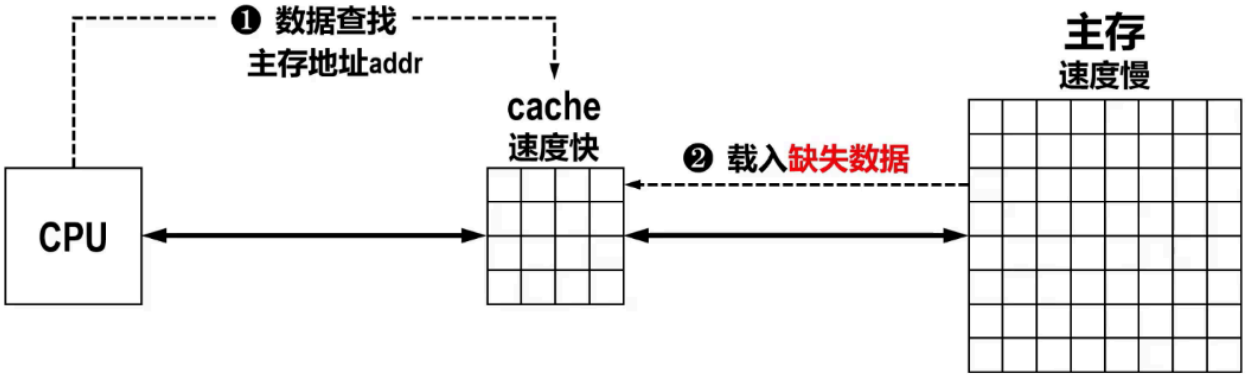
非写分配策略（No-Write-Allocate）



非写分配策略 **此时将数据写入慢速主存即可返回。**

- 写缺失时直接写主存，不把块调入 Cache
- 常与写穿配合
- 优点：避免为一次写而调入整块；缺点：后续读可能缺失

写分配策略（Write-Allocate）



❸ 进行与写命中一样的写入流程（之前刚刚介绍过，就不再赘述了）

写分配(Write-Allocate)策略

- 写缺失时先把块调入 Cache，再在 Cache 中写
- 常与写回配合
- 优点：利用空间局部性；缺点：多一次调入

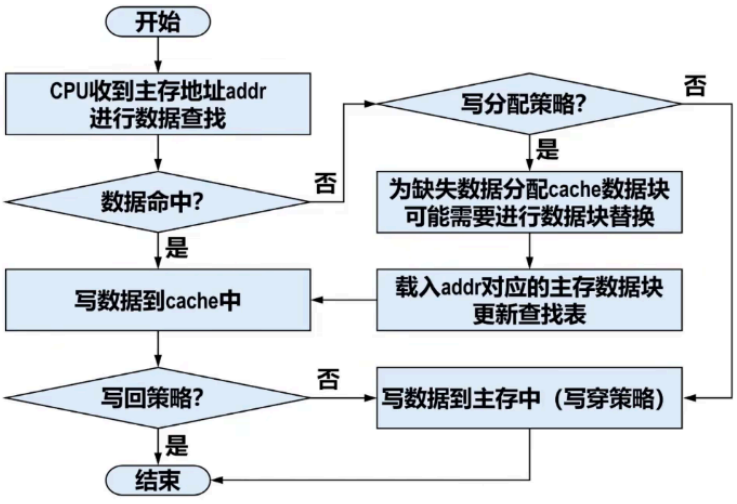
典型组合

- 写回 + 写分配
- 写穿 + 非写分配

易错

- 写分配/非写分配是“写缺失”时的策略，不是“写命中”的策略
- 写回/写穿是“写命中”时的策略
- 四种组合可以交叉，但常见组合只有两种

2.3 写操作流程图



- 采用**写回策略**时，将数据写入cache即可返回，写响应时间最短。

  - 对于**突发的小数据量写入**，cache能明显提高写入性能。

  - 然而，由于cache容量很小，当**cache写满数据后**，需要将cache中的脏数据淘汰，这就需要先将脏数据迁移到主存中，然后从主存载入新数据块到cache后才能向cache写入新数据，该过程的**写性能比没有采用cache的主存还要慢**。

提炼

- 先判断写命中/写缺失
- 命中：选择写回或写穿
- 缺失：选择写分配或非写分配
- 若替换的行是脏的，还要先写回主存

易错

- 写缺失且非写分配时，不需要读主存块
- 写缺失且写分配时，先读块再写，可能触发替换
- 脏位与有效位不是一回事：有效位表示数据是否有效，脏位表示是否与主存一致

知识关联：Cache 基本概念（4.21 Cache——基本概念）；写策略（4.29 Cache——写入策略）；替换算法（4.26 Cache——替换算法）；直接映射（4.23 Cache——地址映射（直接映射））